

# INTRODUCTION TO THE FAST FOURIER TRANSFORM ALGORITHM

# Discrete Fourier Transform

- Evaluate at  $N$  equally spaced Frequencies:

$$\omega_k = \frac{2\pi k}{N} \quad k = 0, 1, 2, \dots, N - 1$$

- N-Point DFT for  $k = 0, 1, 2, \dots, N - 1$  of  $L$  data points of  $x(n)$  is:

$$X(\omega_k) = \sum_{n=0}^{L-1} x(n)e^{-j\omega_k n}$$

- or

$$X(k) = \sum_{n=0}^{L-1} x(n)e^{-\frac{j2\pi kn}{N}}$$

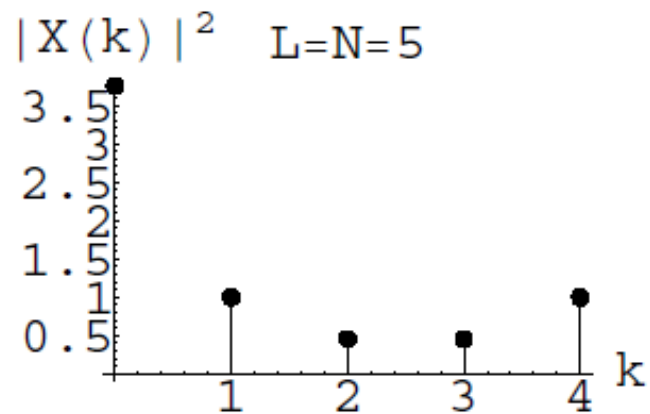
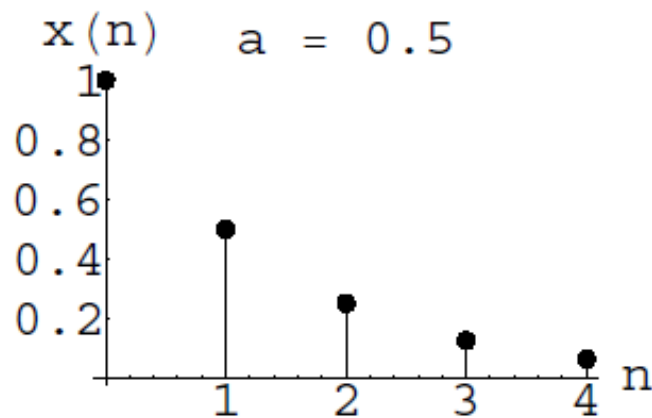


- $L$  and  $N$  can be specified independantly
- $L$  gives the number of data points available from  $x(n)$
- $N$  gives the number of Frequencies that evaluate the DFT.
- Matrix Vector Multiplication  $\Theta(NL)$

$$\begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & e^{-\frac{j2\pi}{N}} & e^{-\frac{j4\pi}{N}} & \dots & e^{-\frac{j2(L-1)\pi}{N}} \\ 1 & e^{-\frac{j4\pi}{N}} & e^{-\frac{j8\pi}{N}} & \dots & e^{-\frac{j4(L-1)\pi}{N}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & e^{-\frac{j2(N-1)\pi}{N}} & e^{-\frac{j4(N-1)\pi}{N}} & \dots & e^{-\frac{j2(L-1)(N-1)\pi}{N}} \end{pmatrix} \begin{pmatrix} x(0) \\ x(1) \\ x(2) \\ \vdots \\ x(L-1) \end{pmatrix} = \begin{pmatrix} X(0) \\ X(1) \\ X(2) \\ \vdots \\ X(N-1) \end{pmatrix}$$

5 point DFT of  $x(n) = 0.5^n$

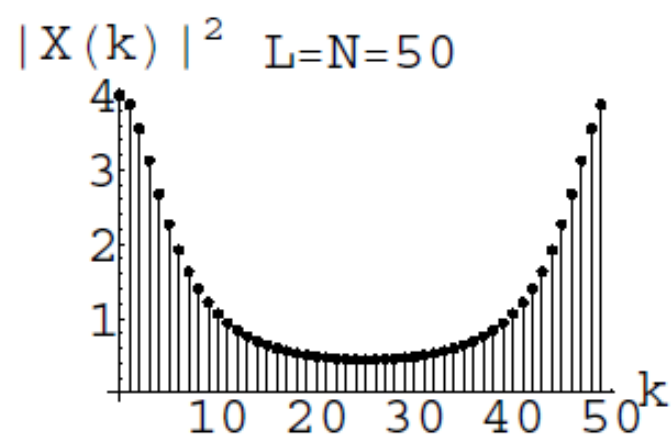
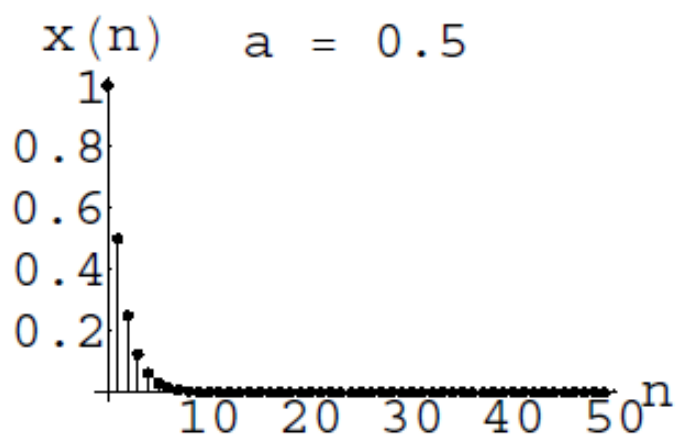
$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & e^{-\frac{j2\pi}{5}} & e^{-\frac{j4\pi}{5}} & e^{-\frac{j6\pi}{5}} & e^{-\frac{j8\pi}{5}} \\ 1 & e^{-\frac{j4\pi}{5}} & e^{-\frac{j8\pi}{5}} & e^{-\frac{j12\pi}{5}} & e^{-\frac{j16\pi}{5}} \\ 1 & e^{-\frac{j6\pi}{5}} & e^{-\frac{j12\pi}{5}} & e^{-\frac{j18\pi}{5}} & e^{-\frac{j24\pi}{5}} \\ 1 & e^{-\frac{j8\pi}{5}} & e^{-\frac{j16\pi}{5}} & e^{-\frac{j24\pi}{5}} & e^{-\frac{j32\pi}{5}} \end{pmatrix} \begin{pmatrix} 1 \\ 0.5 \\ 0.25 \\ 0.125 \\ 0.0625 \end{pmatrix} = \begin{pmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \\ X(4) \end{pmatrix}$$



50 point DFT of  $x(n) = 0.5^n$

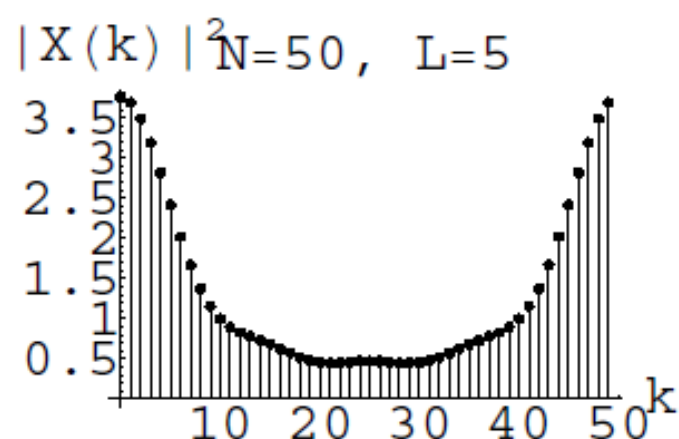
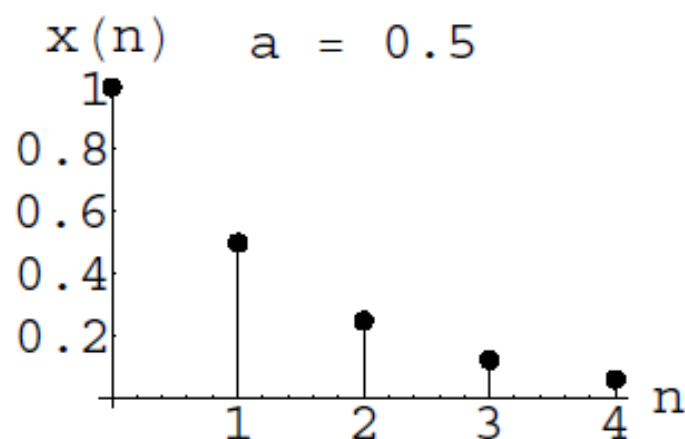
5/28

$$\begin{pmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & e^{-\frac{j2\pi}{50}} & e^{-\frac{j4\pi}{50}} & \dots & e^{-\frac{j98\pi}{50}} \\ 1 & e^{-\frac{j4\pi}{50}} & e^{-\frac{j8\pi}{50}} & \dots & e^{-\frac{j196\pi}{50}} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & e^{-\frac{j98\pi}{50}} & e^{-\frac{j196\pi}{50}} & \dots & e^{-\frac{j4802\pi}{50}} \end{pmatrix} \begin{pmatrix} 1 \\ 0.5 \\ 0.25 \\ \vdots \\ 0.5^{49} \end{pmatrix} = \begin{pmatrix} X(0) \\ X(1) \\ X(2) \\ \vdots \\ X(49) \end{pmatrix}$$



50 point DFT of  $x(n) = 0.5^n$  using 5 data points.

$$\begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 1 & e^{-\frac{j2\pi}{50}} & e^{-\frac{j4\pi}{50}} & e^{-\frac{j6\pi}{50}} & e^{-\frac{j8\pi}{50}} \\ 1 & e^{-\frac{j4\pi}{50}} & e^{-\frac{j8\pi}{50}} & e^{-\frac{j12\pi}{50}} & e^{-\frac{j16\pi}{50}} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & e^{-\frac{j98\pi}{50}} & e^{-\frac{j196\pi}{50}} & e^{-\frac{j294\pi}{50}} & e^{-\frac{j392\pi}{50}} \end{pmatrix} \begin{pmatrix} 1 \\ 0.5 \\ 0.25 \\ 0.125 \\ 0.0625 \end{pmatrix} = \begin{pmatrix} X(0) \\ X(1) \\ X(2) \\ \vdots \\ X(49) \end{pmatrix}$$



Note that  $X(1)$  of the 5 point DFT

$$X(1) = 1 + 0.5e^{-\frac{j2\pi}{5}} + 0.25e^{-\frac{j4\pi}{5}} + 0.125e^{-\frac{j6\pi}{5}} + 0.0625e^{-\frac{j8\pi}{5}}$$

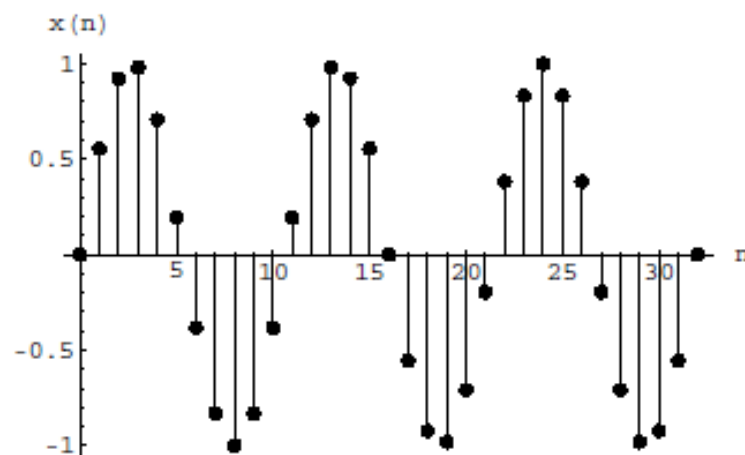
is equivalent to  $X(10)$  of the 50 point DFT using 5 Data points

$$X(10) = 1 + 0.5e^{-\frac{j20\pi}{50}} + 0.25e^{-\frac{j40\pi}{50}} + 0.125e^{-\frac{j60\pi}{50}} + 0.0625e^{-\frac{j80\pi}{50}}$$

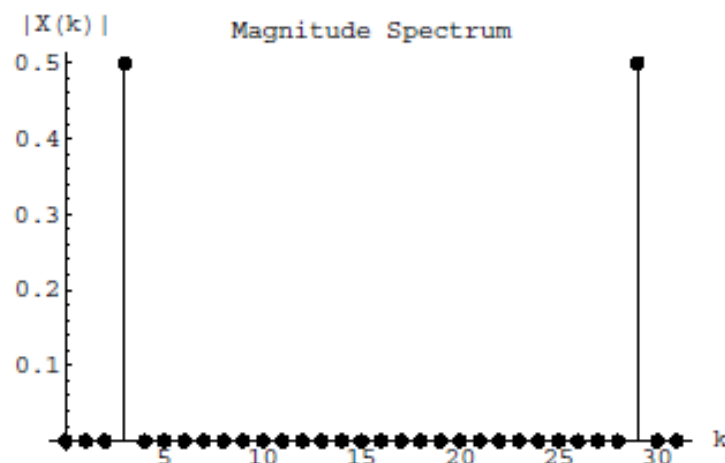
Likewise for  $X_5(0) - X_{50/5}(0)$ ,  $X_5(2) - X_{50/5}(20)$ ,  $X_5(3) - X_{50/5}(30)$  and  $X_5(4) - X_{50/5}(40)$ .

- Plot of  $\sin(2\pi f n)$

- $f = \frac{3}{32}$

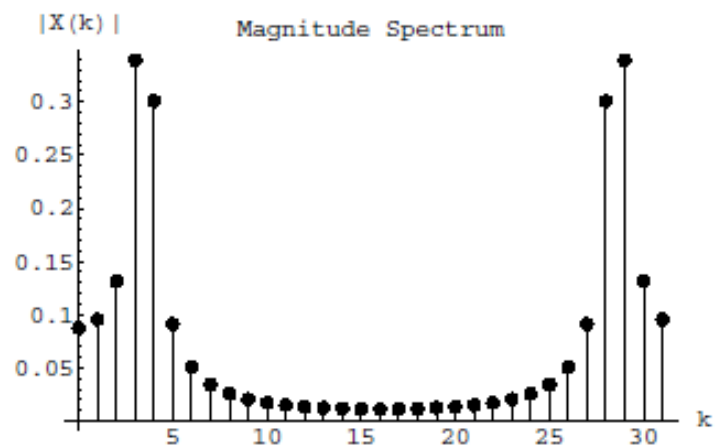
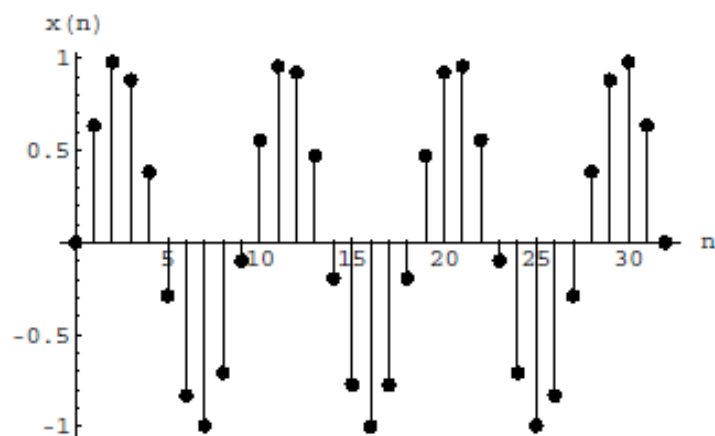


- 32 Point DFT
- Magnitude Spectrum
- Impulse at  $k = 3, 29$





- Plot of  $\sin(2\pi f n)$
- $f = \frac{7}{64}$
- 32 Point DFT
- Magnitude Spectrum
- Power Distributed over  $k = 3, 4$



Fast Fourier Transforms have revolutionized digital signal processing

## What is the FFT?

- A collection of “tricks” that exploit the symmetry of the DFT calculation to make its execution much faster
- Speedup increases with DFT size

## Some dates:

- 1880 - algorithm first described by Gauss
- 1965 - algorithm rediscovered (not for the first time) by Cooley and Tukey

In 1967, calculation of a 8192-point DFT on the top-of-the line IBM 7094 took ....

- 30 minutes using conventional techniques
- 5 seconds using FFTs

## Could consider

- Number of additions
- Number of multiplications
- Amount of memory required
- Scalability and regularity

## For the present discussion we'll focus most on number of multiplications as a measure of computational complexity

- More costly than additions for fixed-point processors
- Same cost as additions for floating-point processors, but number of operations is comparable

**Convolution of an  $N$ -point input with an  $M$ -point unit sample response ....**

**Direct convolution:**

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

- Number of multiplies  $\approx MN$

**Convolution of an  $N$ -point input with an  $M$ -point unit sample response ....**

**Using transforms directly:**

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn / N}$$

- Computation of  $N$ -point DFTs requires  $N^2$  multiplies
- Each convolution requires three DFTs of length  $N+M-1$  plus an additional  $N+M-1$  complex multiplies or

$$3(N + M - 1)^2 + (N + M - 1)$$

- For  $N \gg M$  For example, the computation is  $O(N^2)$

# The Cooley-Tukey decimation-in-time algorithm

- Consider the DFT algorithm for an integer power of 2,

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{nk} = \sum_{n=0}^{N-1} x[n] e^{-j2\pi nk/N}; \quad W_N = e^{-j2\pi/N}$$

- Create separate sums for even and odd values of  $n$ :

$$X[k] = \sum_{n \text{ even}} x[n] W_N^{2k} + \sum_{n \text{ odd}} x[n] W_N^{2k}$$

- Letting  $n = 2r$  for  $n$  even and  $n = 2r + 1$  for  $n$  odd, we obtain

$$X[k] = \sum_{r=0}^{(N/2)-1} x[2r] W_N^{2rk} + \sum_{r=0}^{(N/2)-1} x[2r+1] W_N^{(2r+1)k}$$

Splitting indices in time, we have obtained

$$X[k] = \sum_{r=0}^{(N/2)-1} x[2r] W_N^{2rk} + \sum_{r=0}^{(N/2)-1} x[2r+1] W_N^{(2r+1)k}$$

**But**  $W_N^2 = e^{-j2\pi 2/N} = e^{-j2\pi/(N/2)} = W_{N/2}$  **and**  $W_N^{2rk} W_N^k = W_N^k W_{N/2}^{rk}$

**So ...**

$$X[k] = \sum_{n=0}^{(N/2)-1} x[2r] W_{N/2}^{rk} + W_N^k \sum_{n=0}^{(N/2)-1} x[2r+1] W_{N/2}^{rk}$$

N/2-point DFT of  $x[2r]$       N/2-point DFT of  $x[2r+1]$



## Savings so far ...

**We have split the DFT computation into two halves:**

$$\begin{aligned}
 X[k] &= \sum_{n=0}^{N-1} x[n] W_N^{nk} \\
 &= \sum_{n=0}^{(N/2)-1} x[2r] W_{N/2}^{rk} + W_N^k \sum_{n=0}^{(N/2)-1} x[2r+1] W_{N/2}^{rk}
 \end{aligned}$$

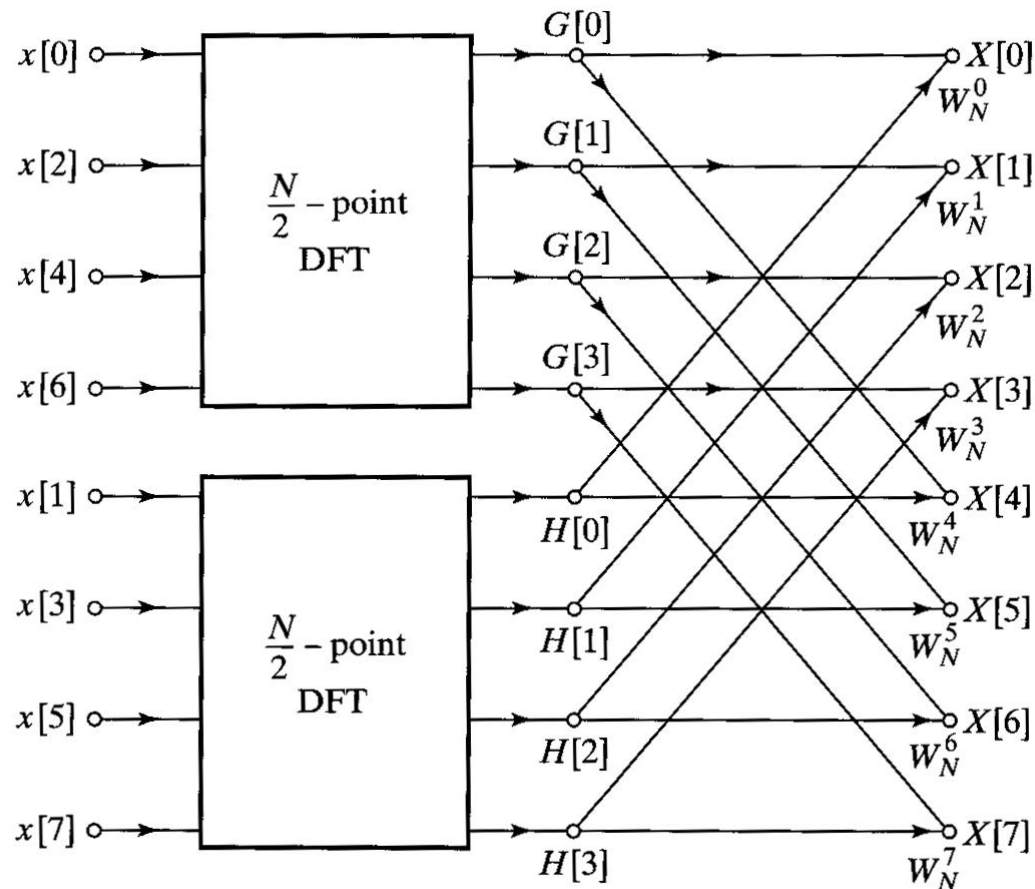
**Have we gained anything? Consider the nominal number of multiplications for  $N = 8$**

- Original form produces  $8^2 = 64$  multiplications
- New form produces  $2(4^2) + 8 = 40$  multiplications
- So we're already ahead ..... Let's keep going!!

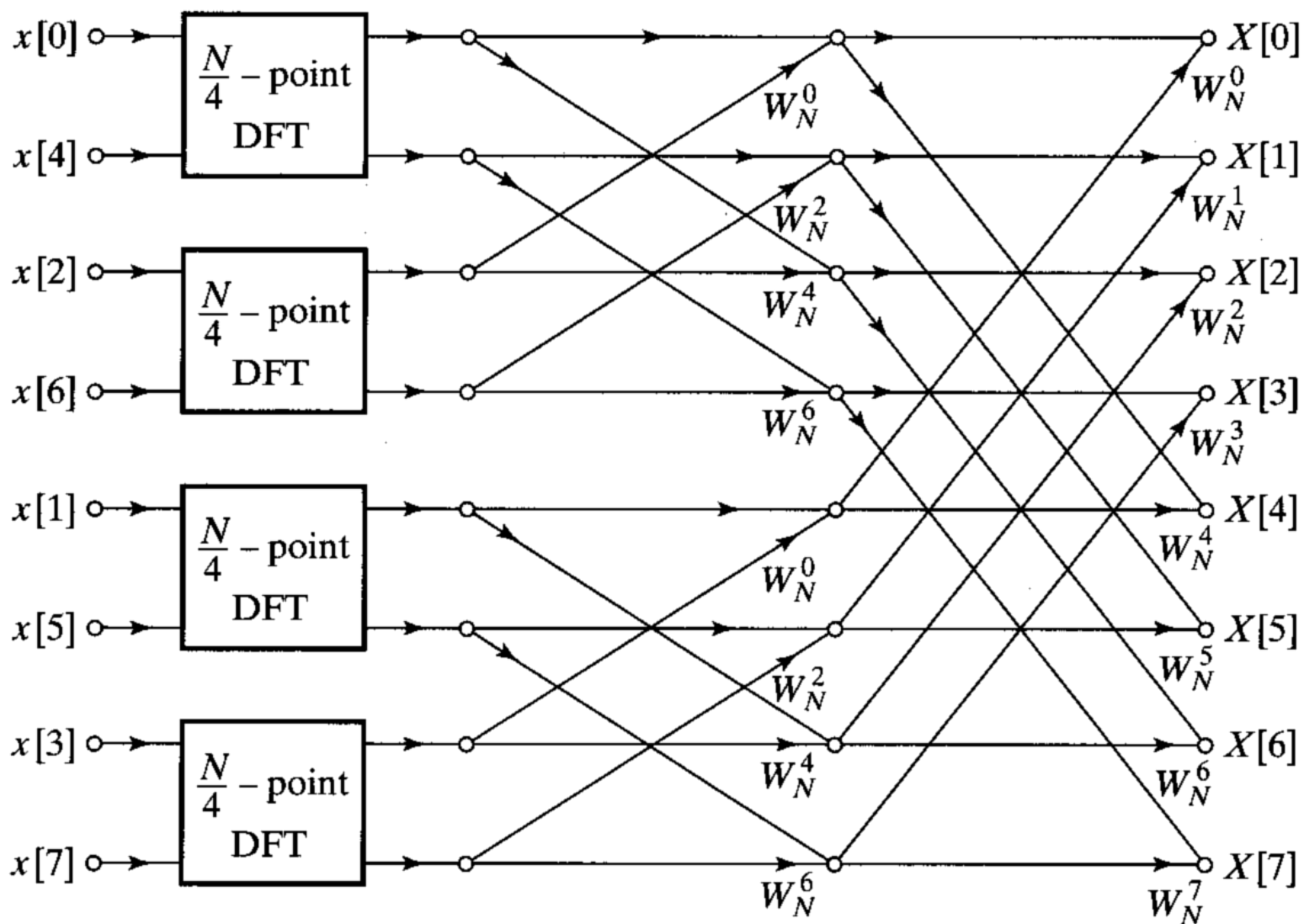
# Signal flowgraph representation of 8-point DFT

Recall that the DFT is now of the form

The DFT in (partial) flowgraph notation:

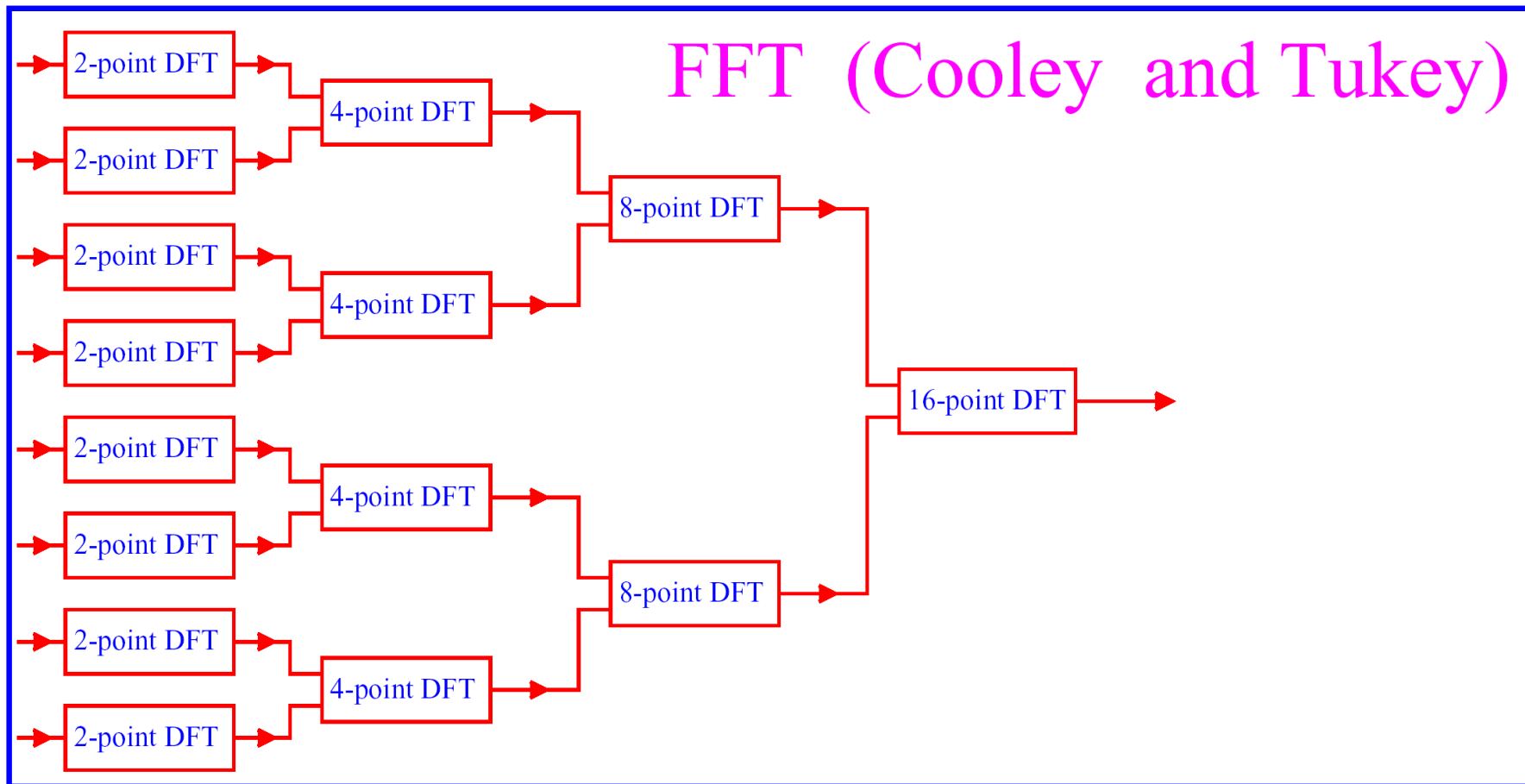


# The complete decomposition into 2-point DFTs



# The complete decomposition into 2-point DFTs

## FFT (Cooley and Tukey)



## Now let's take a closer look at the 2-point DFT

The expression for the 2-point DFT is:

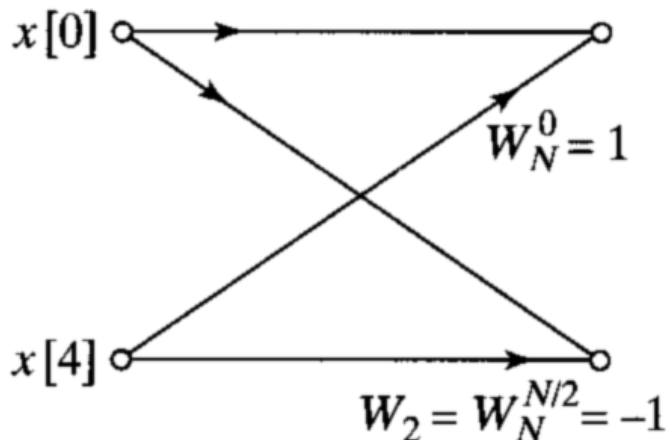
$$X[k] = \sum_{n=0}^1 x[n] W_2^{nk} = \sum_{n=0}^1 x[n] e^{-j2\pi nk/2}$$

Evaluating for  $k = 0, 1$  we obtain

$$X[0] = x[0] + x[1]$$

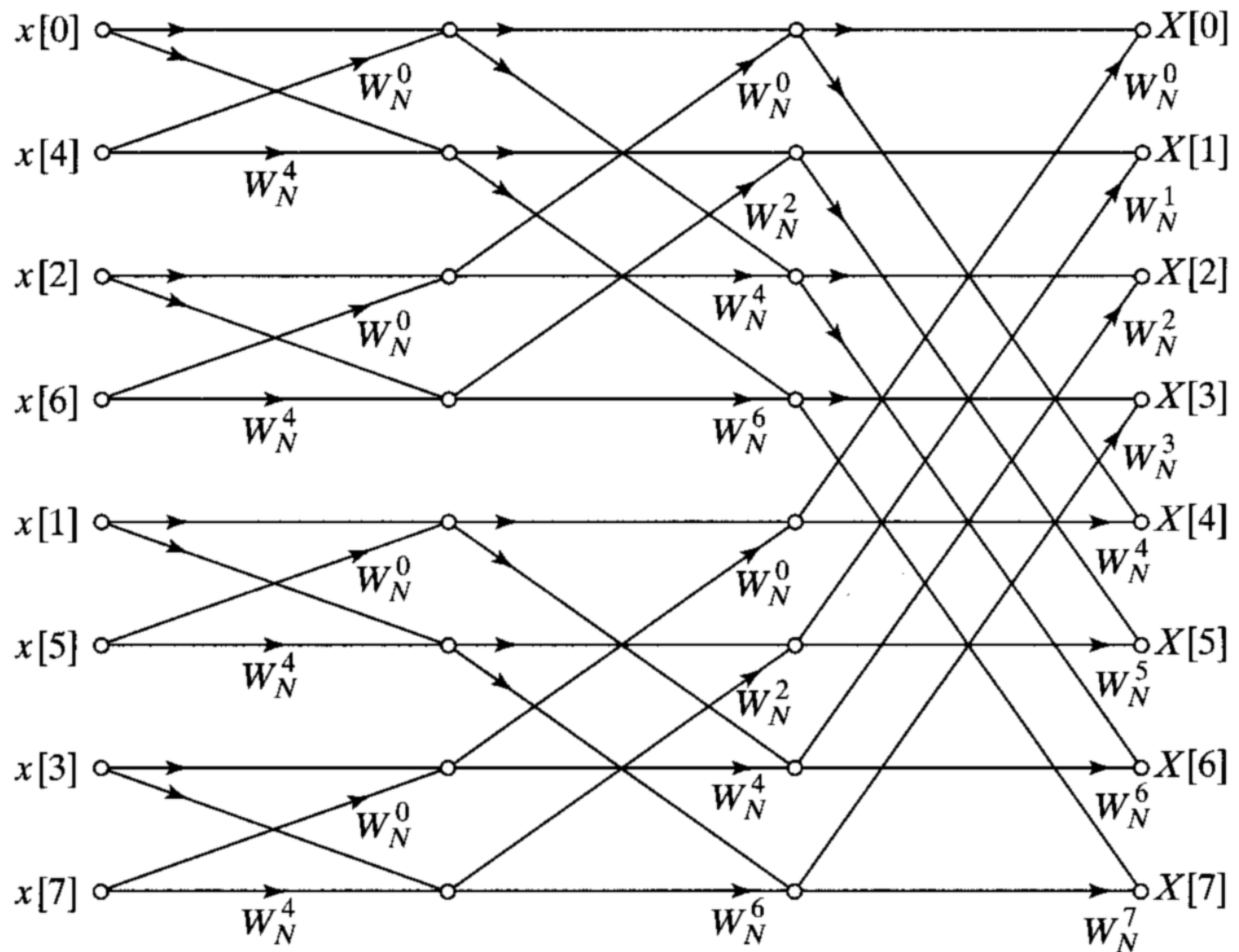
$$X[1] = x[0] + e^{-j2\pi 1/2} x[1] = x[0] - x[1]$$

which in signal flowgraph notation looks like ...

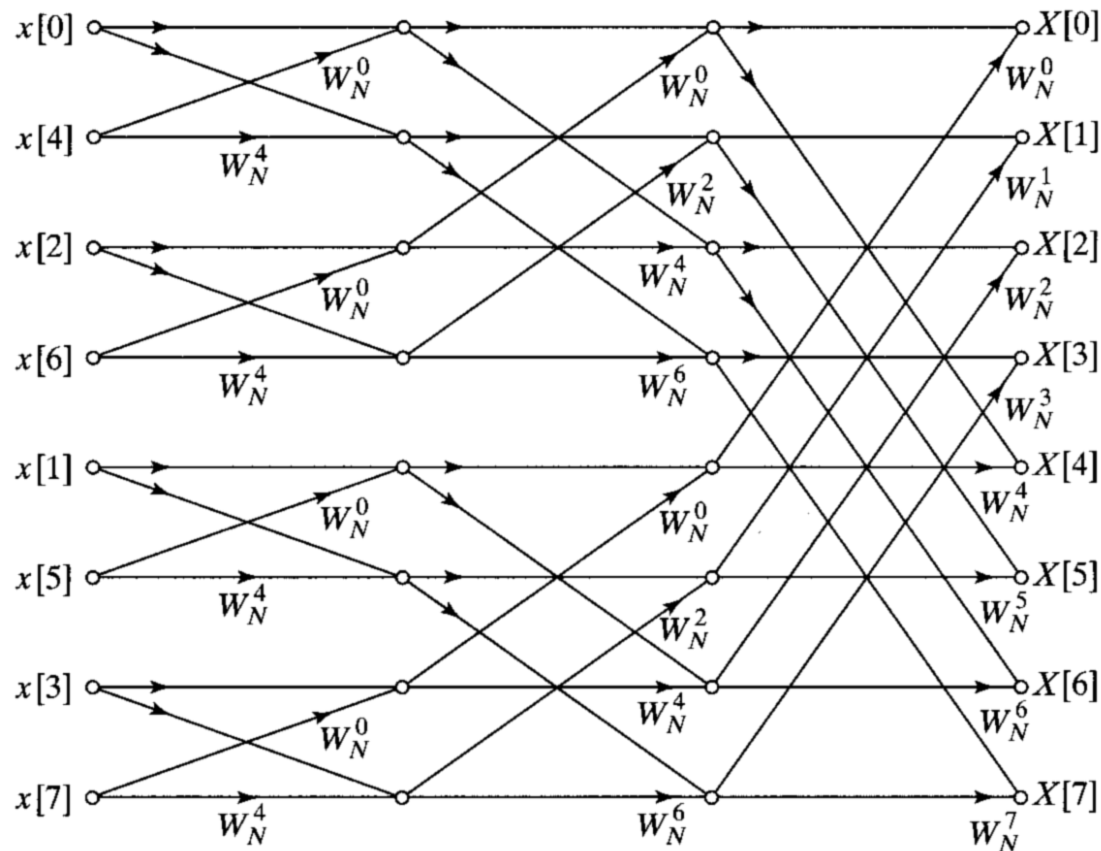


This topology is referred to as the basic butterfly

# The complete 8-point decimation-in-time FFT



# Number of multiplies for N-point FFTs



- Let  $N = 2^\nu$  where  $\nu = \log_2(N)$
- $(\log_2(N) \text{ columns})(N/2 \text{ butterflys/column})(2 \text{ mults/butterfly})$   
or  $\sim N \log_2(N) \text{ multiplies}$

# Computational complexity

Computational complexity: each stage has  $N$  complex multiplications and  $N$  complex additions and there are  $\log_2 N$  stages  $\Rightarrow$  the total complexity (additions and multiplications) is

$$C_{\text{FFT}} = N \log_2 N$$

Compare it with the complexity of the “straightforward” DFT

$$C_{\text{DFT}} = N^2$$

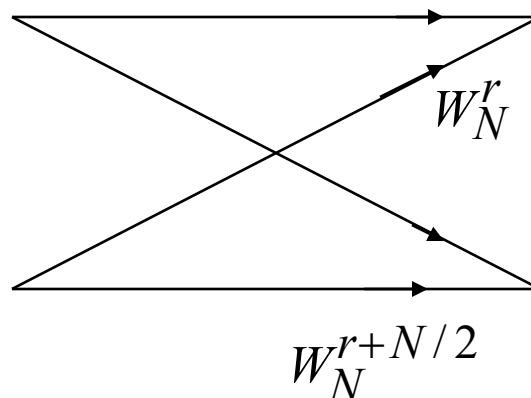
For large  $N$  (i.e.,  $N \gg 1$ ),

$$\log_2 N \ll N \Rightarrow C_{\text{FFT}} \ll C_{\text{DFT}}$$

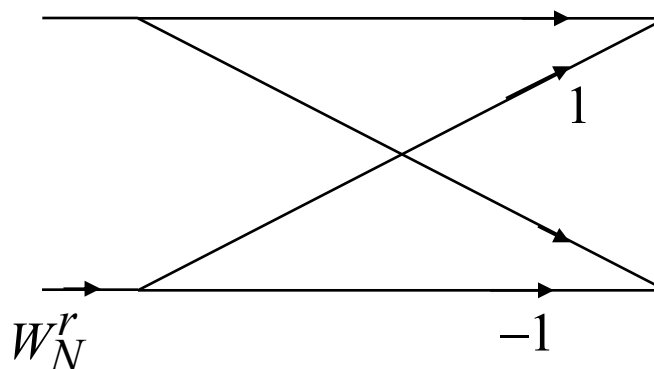
Example: For  $N = 2^{10} = 1024$ ,  $C_{\text{DFT}} = 2^{20} \simeq 10^6$ , whereas  $C_{\text{FFT}} = 10 \cdot 1024 \simeq 10^4$ , i.e., the reduction is about 100 orders of magnitude!



As we derived it, the basic butterfly is of the form

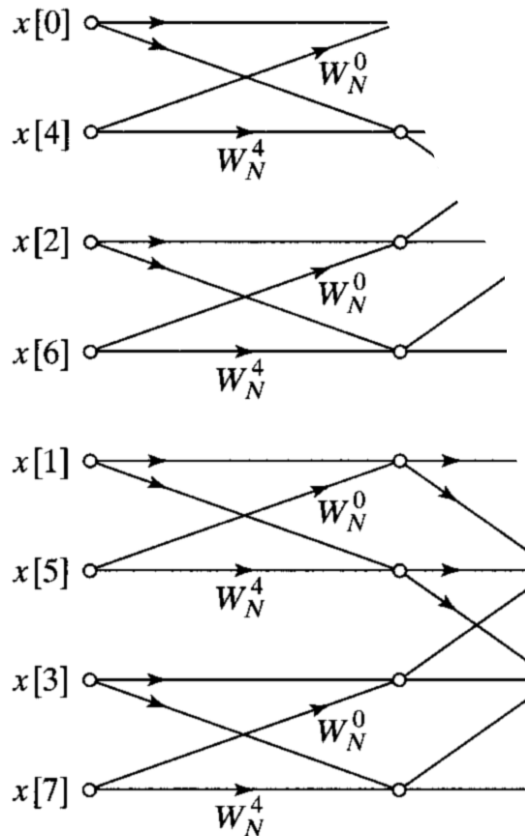


Since  $W_N^{N/2} = 1$  we can reduce computation by 2 by premultiplying by  $W_N^r$



# Bit reversal of the input

Recall the first stages of the 8-point FFT:



Consider the binary representation of the indices of the input:

|   |     |
|---|-----|
| 0 | 000 |
| 4 | 100 |
| 2 | 010 |
| 6 | 110 |
| 1 | 001 |
| 5 | 101 |
| 3 | 011 |
| 7 | 111 |

If these binary indices are time reversed, we get the binary sequence representing 0,1,2,3,4,5,6,7

Hence the indices of the FFT inputs are said to be in **bit-reversed order**

**In the implementation of the FFT that we discussed, the input is bit reversed and the output is developed in natural order**

**Some other implementations of the FFT have the input in natural order and the output bit reversed**

**In some situations it is convenient to implement filtering applications by**

- Use FFTs with input in natural order, output in bit-reversed order
- Multiply frequency coefficients together (in bit-reversed order)
- Use inverse FFTs with input in bit-reversed order, output in natural order

**Computing in this fashion means we never have to compute bit reversal explicitly**

# Decimation in Freq FFT

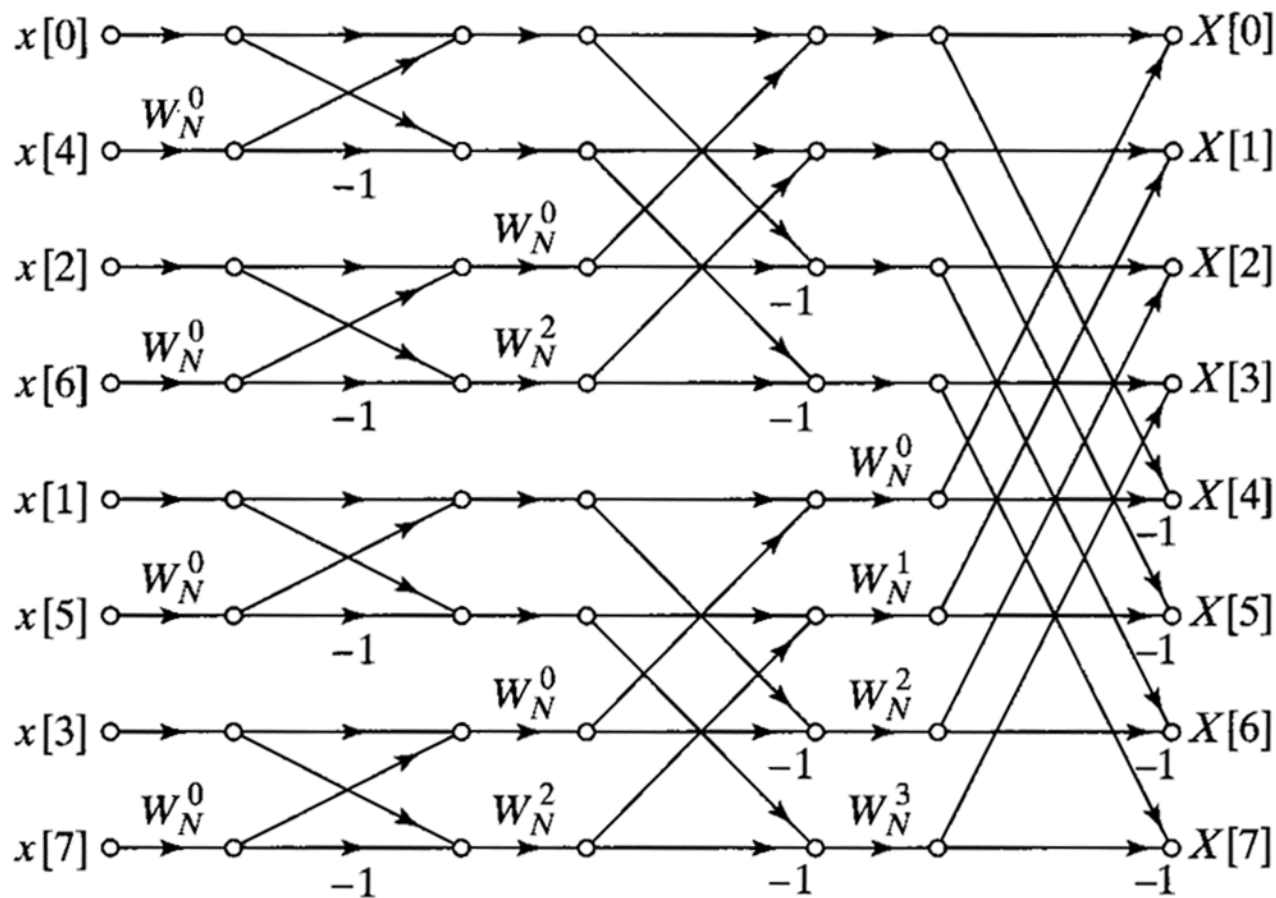
Principle: separate  $X[k]$  into even- and odd-numbered components:

$$\begin{aligned}
 X[2k] &= \sum_{n=0}^{N-1} x[n] W_N^{2kn} \\
 &= \sum_{n=0}^{N/2-1} x[n] W_N^{2kn} + \sum_{n=N/2}^{N-1} x[n] W_N^{2kn} \\
 &= \sum_{n=0}^{N/2-1} (x[n] + x[n + N/2]) W_{N/2}^{rn} \\
 X[2k + 1] &= \sum_{n=0}^{N-1} x[n] W_N^{(2k+1)n} \\
 &= \sum_{n=0}^{N/2-1} x[n] W_N^{(2k+1)n} + \sum_{n=N/2}^{N-1} x[n] W_N^{(2k+1)n} \\
 &= \sum_{n=0}^{N/2-1} (x[n] - x[n + N/2]) W_N^n W_{N/2}^{rn}
 \end{aligned}$$

- ◆ We developed the basic decimation-in-time (DIT) FFT structure, but other forms are possible simply by rearranging the branches of the signal flowgraph
- ◆ Consider the rearranged signal flow diagrams on the following panels .....

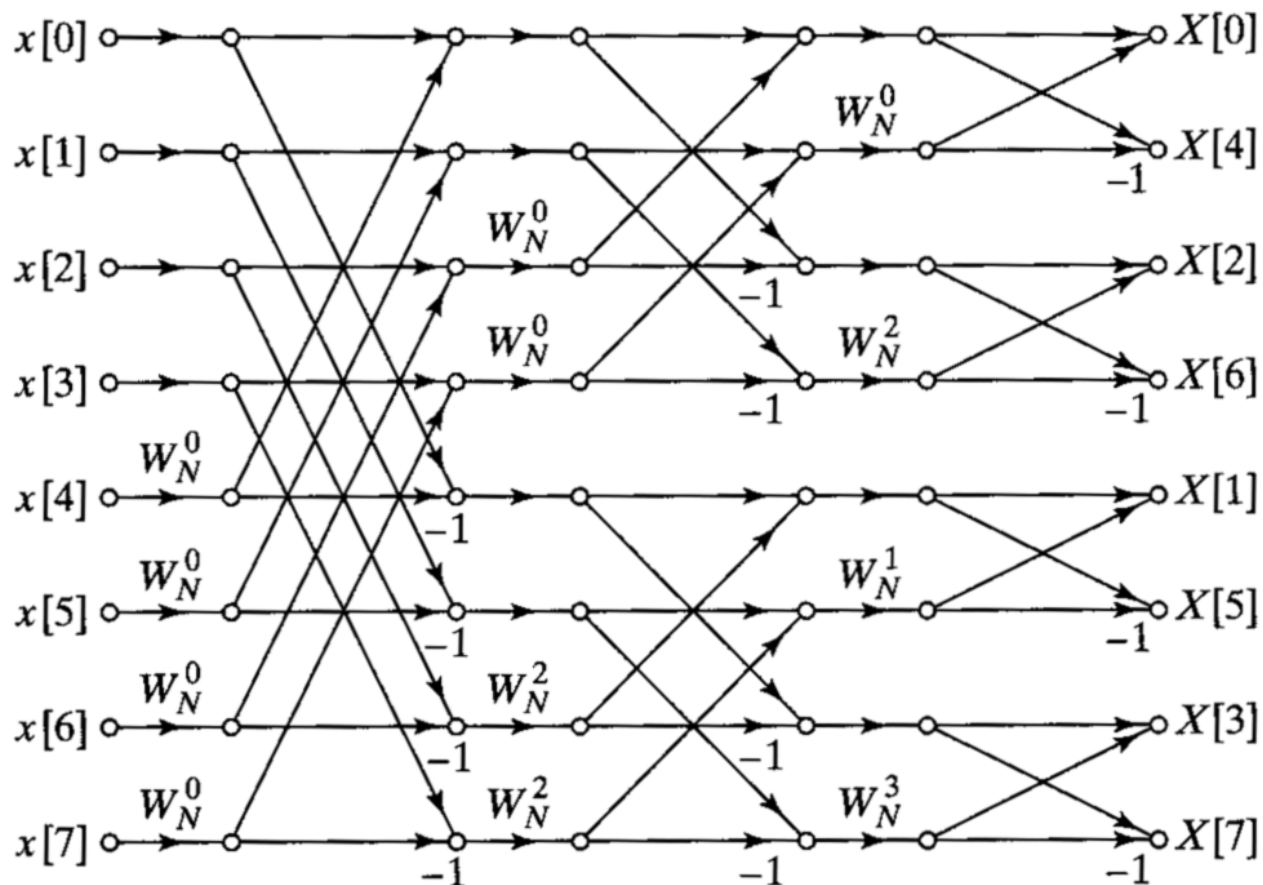
# Alternate DIT FFT structures (continued)

- ◆ DIT structure with input bit-reversed, output natural (OSB 9.10):



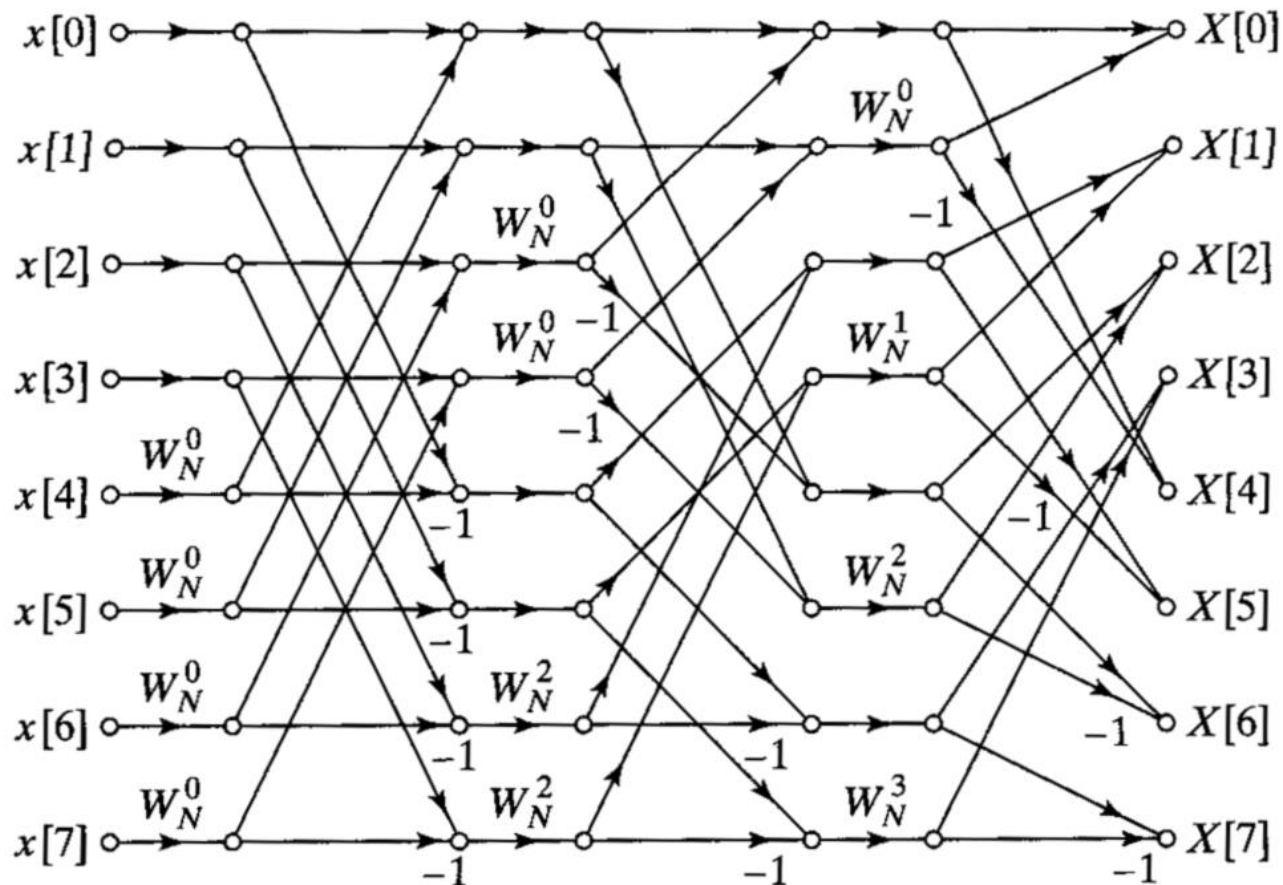
# Alternate DIT FFT structures (continued)

- ◆ DIT structure with input natural, output bit-reversed (OSB 9.14):



# Alternate DIT FFT structures (continued)

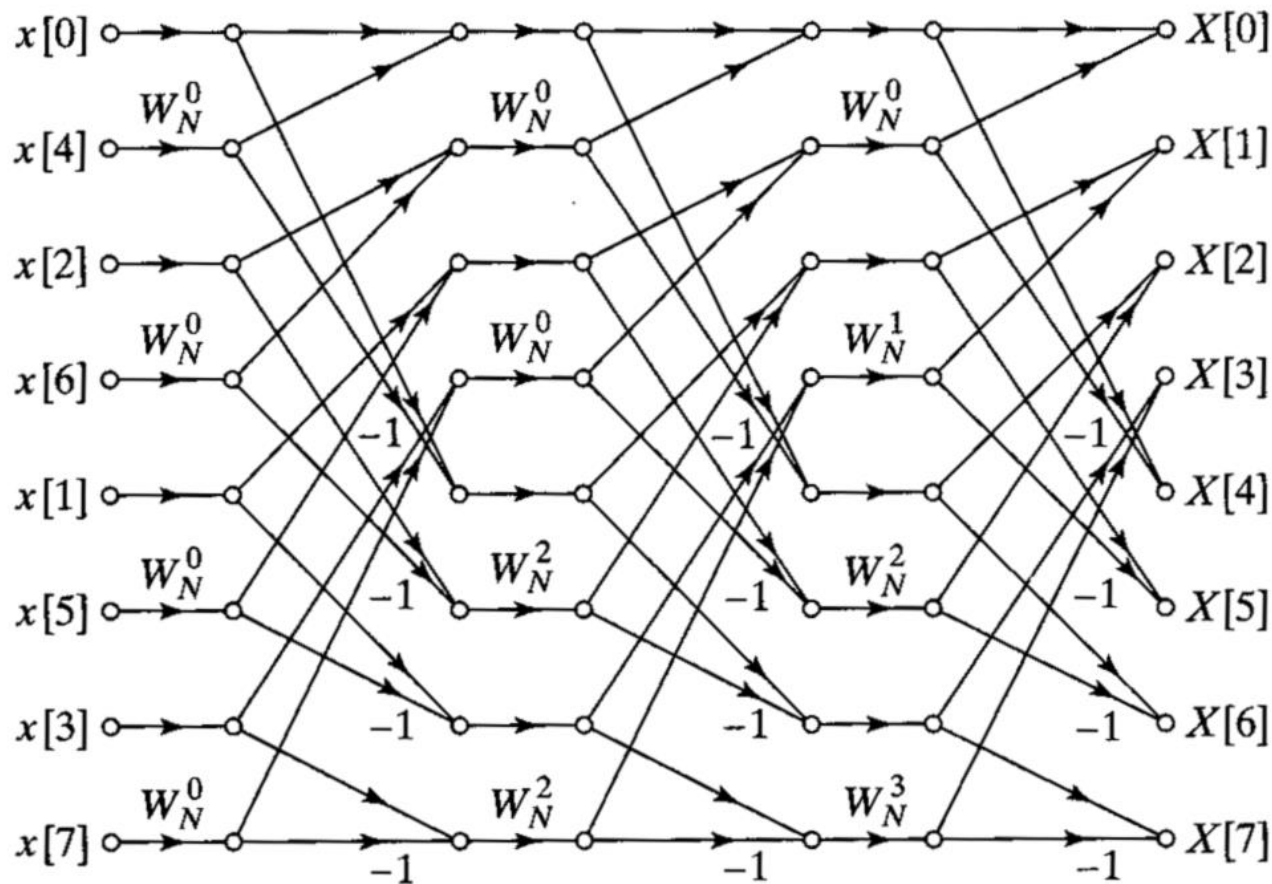
- ◆ DIT structure with both input and output natural (OSB 9.15):





## Alternate DIT FFT structures (continued)

- ◆ DIT structure with same structure for each stage (OSB 9.16):



- ◆ A method to avoid bit-reversal in filtering operations is:
  - Compute forward transform using natural input, bit-reversed output (as in OSB 9.10)
  - Multiply DFT coefficients of input and filter response (both in bit-reversed order)
  - Compute inverse transform of product using bit-reversed input and natural output (as in OSB 9/14)
- ◆ Latter two topologies (as in OSB 9.15 and 9.16) are now rarely used

## Using FFTs for inverse DFTs

- ◆ We've always been talking about forward DFTs in our discussion about FFTs .... what about the inverse FFT?

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] W_N^{-kn}; \quad X[k] = \sum_{n=0}^{N-1} x[n] W_N^{kn}$$

- ◆ One way to modify FFT algorithm for the inverse DFT computation is:

- Replace  $W_N^k$  by  $W_N^{-k}$  wherever it appears
- Multiply final output by  $1/N$

- ◆ This method has the disadvantage that it requires modifying the internal code in the FFT subroutine

**Taking the complex conjugate of both sides of the IDFT equation and multiplying by  $N$ :**

$$Nx^*[n] = \frac{1}{N} \sum_{k=0}^{N-1} X^*[k] W_N^{kn}; \text{ or } x[n] = \frac{1}{N} \left[ \sum_{k=0}^{N-1} X^*[k] W_N^{kn} \right]^*$$

**This suggests that we can modify the FFT algorithm for the inverse DFT computation by the following:**

- Complex conjugate the input DFT coefficients
- Compute the *forward* FFT
- Complex conjugate the output of the FFT and multiply by  $1/N$

**This method has the advantage that the internal FFT code is undisturbed; it is widely used.**